

 IBM Bob 2.0 Hackathon Submission

UpgradePilot

AI-powered agentic co-pilot that scans, plans, applies, and verifies dependency upgrades — automated, diff-reviewed, and test-gated.

Node.js + Python

6-Agent Pipeline

67 / 67 Tests ✓

81% Time Saved

Built by IBM Bob 2.0



Scanner



Librarian



Planner



Codemoder



Verifier



Scribe

THE PROBLEM

Dependency upgrades are a developer nightmare



Massive time sink

Upgrading one major-version dep (e.g. Express 4→5) means grepping source code, reading changelogs, writing codemods, and fixing test regressions manually.

~20 hours



Error-prone & incomplete

Manual grep misses hits in edge cases, comments, and dynamic calls. Partial migrations leave silent regressions that only surface in production.

53+ hits missed



Multi-ecosystem complexity

Teams run Node.js and Python services. Each ecosystem has different tooling, patterns, and migration guides — no unified workflow exists.

2× toolchains



Blocks velocity

Outdated deps accumulate. Security patches, EOL migrations, and framework upgrades pile up into multi-sprint refactors that derail roadmaps.

Sprints lost

THE SOLUTION

One CLI. Scan → Plan → Fix → Verify.

UpgradePilot is a six-agent pipeline that automates the entire upgrade workflow — from detecting breaking patterns to confirming green tests on new majors.

STEP 01

 Scan

Auto-detects Node.js or Python. Globs all source files. Matches 18 breaking-change rules line-by-line with exact file:line hits and risk scores.

18 rules · 5 packages

STEP 02

 Plan

Risk formula: occurrence × severity × proximity weight. Safe upgrade order: dev deps first, then prod ascending. Manual vs pilot effort on every step.

Risk · Order · Time

STEP 03

 Fix

Promise.all worker-per-package codemod crew. Pure-JS unified differ. Generates .patch files and replacement log. Zero source file writes in dry-run.

Safe dry-run

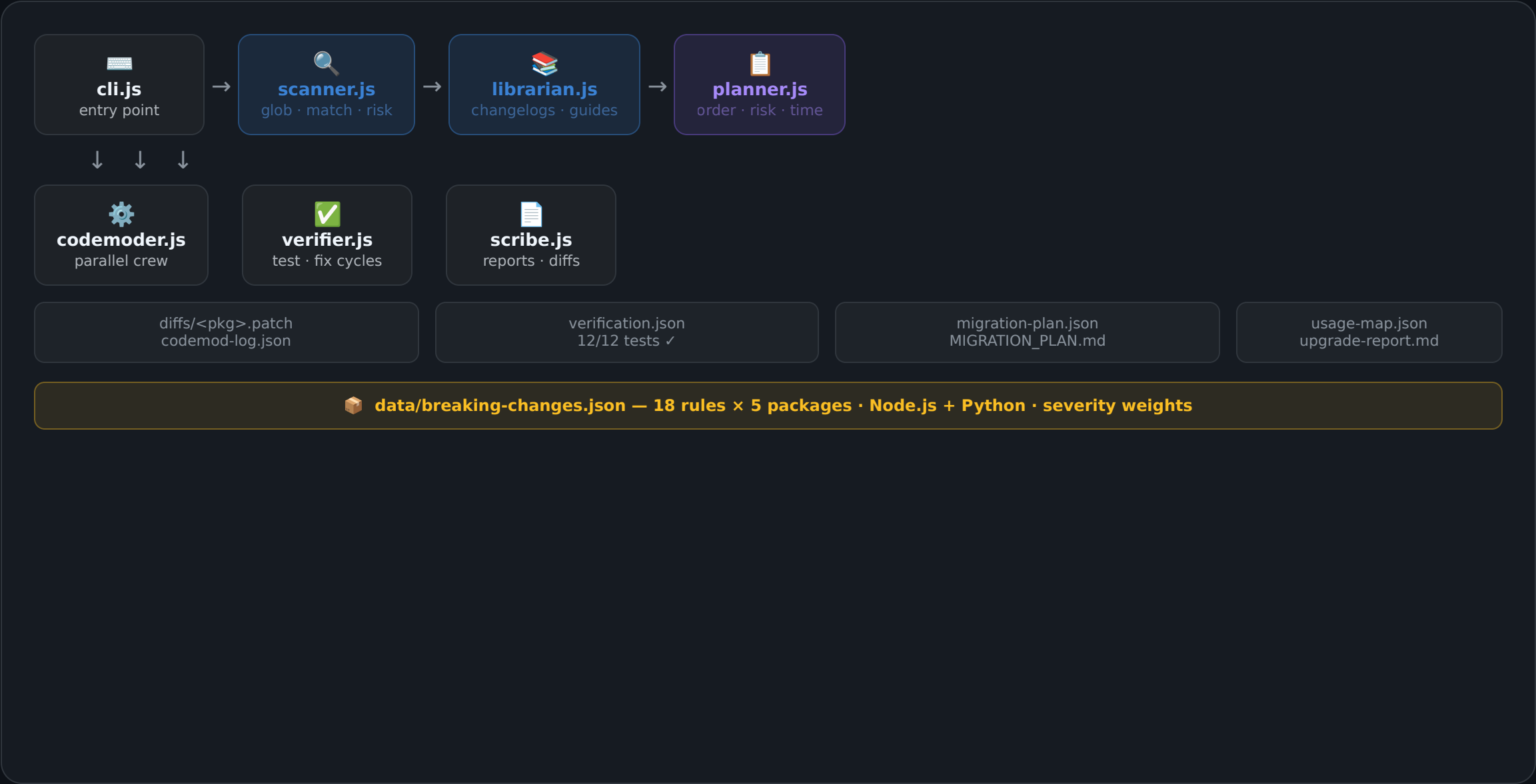
STEP 04

 Verify

Bumps versions, runs install + test, applies up to 3 targeted fix cycles if tests fail, writes verification.json with final counts.

12/12 green · 0 cycles

Six-Agent Pipeline



Five subcommands. Two ecosystems.

1

scan — Detect & risk-score breaking patterns

Auto-detects Node.js (.js) or Python (.py + requirements.txt). 18 rules matched line-by-line. Exact file:line hits with severity-weighted risk scores.

Express 4→5

Mongoose 6→7

Jest 27→29

Flask 2→3

requests 2.28→2.32

2

plan — Risk-scored migration plan with effort estimates

Occurrence × severity × proximity formula. Dev deps first, then prod ascending. Manual vs UpgradePilot estimate on every step.

1,208 min → 228 min (Node)

216 min → 41 min (Python)

3

run — Parallel codemod crew + unified diff patches

Promise.all per package. Pure-JS differ — no external dep. Dry-run writes .patch files + log but zero source files. Ecosystem-aware (.js or .py).

0 source files in --dry-run

3 .patch files generated

4

demo — Both ecosystems end-to-end in one command

`node tool/cli.js demo` → full scan → plan → codemod → verify → report on both sample apps. Exit 0.

Node.js report ✓

Python report ✓

MEASURED IMPACT

Numbers don't lie

81%

Time saved on
Node.js migration

12/12

Tests green on
new majors · 0 fix cycles

67

Unit tests
all passing

0

Legacy source hits
after migration

Node.js — Express 4→5 + Mongoose 6→7 + Jest 27→29

Manual 1,208 min (~20h)

UpgradePilot 228 min

53 breaking hits detected · 8 rules · plan in <1 s

Python — Flask 2→3 + requests 2.28→2.32

Manual 216 min (~3.6h)

UpgradePilot 41 min

13 hits · flask (high 66/100) · requests (low 13/100)

BUILT WITH IBM BOB 2.0

7 task sessions. Zero manual edits.

Scaffold: Monorepo + Legacy Sample App

Session 1

Monorepo, CLI skeleton with 6 agent stubs, Acme Orders API with 12 deliberate breaking patterns.

✓ 12/12 tests on legacy deps · --help prints clean

Scanner: Usage Map + Risk Scoring

Session 2

Full source scanner, breaking-changes.json rule DB, usage-map.json, 20 unit tests.

✓ 8 rules · exact file:line hits · tests pass

Planner: Risk-Scored Migration Plan

Session 3

Occurrence × severity × proximity formula, effort estimates, Markdown renderer, ARCHITECTURE.md.

✓ migration-plan.json + MIGRATION_PLAN.md generated

Codemod Crew: Parallel Migrations

Session 4

Promise.all worker-per-package, pure-JS unified differ, codemod-log.json, 25 unit tests.

✓ 3 patches · 0 file writes in dry-run · tests pass

Verifier: Tests to Green

Session 5

Real-mode verifier with fix cycles. Migrated sample-app to Express 5 + Mongoose 7.

✓ 12/12 green on new majors · 0 fix cycles

Polish + Python Generalization

Sessions 6-7

CI workflow, README, demo script. Extended all agents for Python. 22 new tests. CI fixed.

✓ 67/67 tests · demo runs both ecosystems · CI green



IBM Bob 2.0 Agent mode throughout — surgical apply_diff, parallel reads, live todo tracking, CLI validation after every step. Architecture designed for **IBM watsonx.ai Granite** injection: each agent exposes a documented LLM-replaceable reasoning step.

TRY IT

Stop dreading upgrades. Let the pilot fly.

81%

Time saved

67

Tests passing

2

Ecosystems

0

Manual edits

⚡ [node tool/cli.js demo](#)

📦 github.com/dragunxwho13/Upgrade-Pilot

Built entirely with **IBM Bob 2.0** in Agent mode · Designed for **IBM watsonx.ai Granite**